

IoT-Based Data Logger with Threshold Violation Detection

LPC2148 | DHT11 | ESP01 WiFi | ThingSpeak Cloud

Project Report — Keil uVision & Proteus Design Suite
Prepared: June 2026

Component	Model / Part	Interface	Role
Microcontroller	LPC2148 (ARM7TDMI-S)	—	Core processing unit
Temp/Humidity	DHT11 Sensor	Single-wire I/O	Environmental sensing
Display	16x2 LCD (HD44780)	GPIO 8-bit	Real-time display
Alert Output	Buzzer / LED	GPIO	Threshold alert
User Input	4x4 Matrix Keypad	GPIO	Set thresholds
Non-volatile Mem	EEPROM AT24C256	I2C	Store setpoints
Real-Time Clock	DS1307 RTC	I2C	Timestamp logging
Ext Interrupt	Push Switch	EINT0	Event trigger
WiFi / Cloud	ESP01 (ESP8266)	UART 9600	ThingSpeak upload

Table 1: System Components Overview

1. Introduction

This project implements a compact IoT data logger that continuously measures temperature and humidity using a DHT11 sensor interfaced to an LPC2148 ARM7 microcontroller. When values exceed user-configured thresholds, a buzzer/LED alerts locally. All readings are uploaded to ThingSpeak cloud via an ESP01 (ESP8266) WiFi module over UART, enabling remote monitoring from any internet-connected device.

Threshold setpoints are stored in an AT24C256 EEPROM over I2C, surviving power cycles. A 4x4 keypad handles local configuration; a 16x2 LCD shows live readings. A DS1307 RTC timestamps each log entry. An EINT0 push-switch triggers on-demand snapshot logging. Firmware is developed in Embedded C on Keil uVision; simulation runs in Proteus 8.

1.1 Objectives

- Continuously sense temperature and humidity with DHT11.
- Display live data on 16x2 LCD with RTC timestamp.
- Alert via buzzer/LED on threshold violation.
- Persist configurable thresholds in EEPROM (AT24C256, I2C).
- Upload data to ThingSpeak every 15 s via ESP01 UART.
- Allow local threshold configuration through 4x4 matrix keypad.
- Simulate and verify the full design in Proteus before hardware assembly.

2. System Architecture

The LPC2148 is the central processing unit. Peripheral communication follows standard embedded protocols: single-wire for DHT11, I2C for EEPROM and RTC, UART for ESP01, and GPIO for LCD, keypad, buzzer, LED, and switch.

2.1 Data Flow

- DHT11 sends 40-bit serial data (16-bit temp + 16-bit hum + 8-bit checksum) to P0.10.
- LPC2148 decodes data, compares against EEPROM thresholds, updates LCD, drives buzzer/LED.
- ESP01 performs HTTP GET to ThingSpeak with field1=temp & field2=hum URL parameters.
- AT24C256 EEPROM stores thresholds at address 0x0000 (temp) and 0x0001 (hum) via I2C.
- DS1307 RTC provides real-time timestamp over I2C for each log entry.
- Keypad lets user navigate menu and enter new threshold values; '#' confirms entry.

2.2 LPC2148 Pin Allocation

LPC2148 Pin(s)	Peripheral	Protocol	Dir
P0.0 (UART0 TXD)	ESP01 RXD	UART 9600 baud	OUT
P0.1 (UART0 RXD)	ESP01 TXD	UART 9600 baud	IN
P0.2 (SDA)	EEPROM + RTC SDA	I2C bit-bang	Bi
P0.3 (SCL)	EEPROM + RTC SCL	I2C bit-bang	OUT
P0.10	DHT11 Data	Single-wire	Bi
P0.14 (EINT1)	Push Switch	External Intr	IN
P1.16 – P1.23	LCD D0–D7	GPIO 8-bit	OUT
P1.24 / P1.25 / P1.26	LCD RS / RW / EN	GPIO	OUT
P0.20	Buzzer	GPIO	OUT
P0.21	Alert LED	GPIO	OUT
P0.22 – P0.29	Keypad Row/Col	GPIO matrix	Bi

Table 2: LPC2148 Pin Allocation

3. Keil uVision Firmware (Embedded C — main.c)

Complete production-ready source code. Build target: LPC2148, ARMCC compiler. Replace YourSSID / YourPass with your WiFi credentials and YOUR_API_KEY_HERE with your ThingSpeak Write API Key before compiling.

3.1 File Header & Includes

```
/* *****  
 * Project   : IoT Data Logger with Threshold Violation Detection  
 * MCU       : LPC2148 (ARM7TDMI-S) @ 60 MHz  
 * Compiler  : Keil uVision 4/5 (ARMCC)  
 * Date      : 2025  
 * Author    : Rajashekar Miryala  
 * ***** */  
  
#include <lpc214x.h>  
#include <stdint.h>  
#include <string.h>  
#include <stdio.h>
```

3.2 Defines & Globals

```
/* =====  
 * CLOCK DEFINES  
 * ===== */  
#define FOSC      12000000UL  
#define PLL_MUL   5  
#define CCLK      (FOSC * PLL_MUL)    /* 60 MHz */  
#define PCLK      (CCLK / 4)          /* 15 MHz */  
  
/* =====  
 * DHT11 (P0.10 - single wire)  
 * ===== */  
#define DHT11_PIN (1UL << 10)  
  
/* =====  
 * LCD (8-bit mode)  
 * Data : P1.16 - P1.23  
 * RS   : P1.24   RW : P1.25   EN : P1.26  
 * ===== */  
#define LCD_RS      (1UL << 24)  
#define LCD_RW      (1UL << 25)  
#define LCD_EN      (1UL << 26)  
#define LCD_DATA_MASK 0x00FF0000UL  
  
/* =====  
 * BUZZER P0.20   ALERT LED P0.21  
 * ===== */  
#define BUZZER      (1UL << 20)  
#define ALERT_LED   (1UL << 21)  
  
/* =====  
 * I2C (bit-bang) SDA=P0.2   SCL=P0.3  
 * ===== */  
#define SDA_PIN     (1UL << 2)  
#define SCL_PIN     (1UL << 3)  
#define EEPROM_ADDR 0xA0    /* AT24C256 */  
#define RTC_ADDR    0xD0    /* DS1307 */  
  
/* =====  
 * UART0 (ESP01 WiFi) 9600 baud  
 * ===== */  
#define BAUD_RATE    9600  
#define THINGSPEAK_KEY "YOUR_API_KEY_HERE"  
  
/* =====  
 * GLOBAL SENSOR VALUES & THRESHOLDS  
 * ===== */  
volatile uint8_t g_temp      = 0;  
volatile uint8_t g_hum       = 0;  
volatile uint8_t g_thresh_temp = 35; /* default 35 C */  
volatile uint8_t g_thresh_hum  = 80; /* default 80 % */
```

3.3 Delay & PLL Init

```
/* =====  
 * DELAY FUNCTIONS  
 * ===== */  
void delay_us(uint32_t us)
```

```

{
    uint32_t i;
    for (i = 0; i < (us * (CCLK / 1000000UL) / 5); i++)
        __nop();
}

void delay_ms(uint32_t ms)
{
    while (ms--)
        delay_us(1000);
}

/* =====
 * PLL INIT -> CPU = 60 MHz
 * ===== */
void pll_init(void)
{
    PLLCON = 0x01;          /* Enable PLL */
    PLLCFG = 0x24;          /* M=5, P=2 -> 60 MHz */
    PLLFEED = 0xAA;
    PLLFEED = 0x55;
    while (!(PLLSTAT & (1 << 10))); /* wait PLL lock */
    PLLCON = 0x03;          /* Enable + Connect */
    PLLFEED = 0xAA;
    PLLFEED = 0x55;
    VPBDIV = 0x00;          /* PCLK = CCLK/4 = 15 MHz */
}

```

3.4 LCD Driver

```

/* =====
 * LCD DRIVER (8-bit, P1 port)
 * ===== */
void lcd_pulse_en(void)
{
    IOSET1 = LCD_EN;
    delay_us(2);
    IOCLR1 = LCD_EN;
    delay_us(2);
}

void lcd_write(uint8_t data, uint8_t rs)
{
    IODIR1 |= LCD_DATA_MASK | LCD_RS | LCD_RW | LCD_EN;
    IOCLR1 = LCD_DATA_MASK | LCD_RS | LCD_RW;
    if (rs)
        IOSET1 = LCD_RS;
    IOSET1 = ((uint32_t)data << 16) & LCD_DATA_MASK;
    lcd_pulse_en();
    delay_us(100);
}

void lcd_cmd(uint8_t cmd)
{
    lcd_write(cmd, 0);
    delay_ms(2);
}

void lcd_data(uint8_t d)
{
    lcd_write(d, 1);
}

void lcd_init(void)
{
    delay_ms(20);
    lcd_cmd(0x38); /* Function set: 8-bit, 2 lines, 5x8 */
    lcd_cmd(0x0C); /* Display ON, cursor OFF */
    lcd_cmd(0x06); /* Entry mode: increment, no shift */
    lcd_cmd(0x01); /* Clear display */
}

void lcd_pos(uint8_t row, uint8_t col)
{
    lcd_cmd(row ? (0xC0 + col) : (0x80 + col));
}

void lcd_str(const char *s)
{

```

```

    while (*s)
        lcd_data((uint8_t)*s++);
}

void lcd_num(uint8_t n)
{
    lcd_data('0' + n / 10);
    lcd_data('0' + n % 10);
}

```

3.5 DHT11 Driver

```

/* =====
 * DHT11 SINGLE-WIRE DRIVER
 * ===== */
uint8_t dht11_read(uint8_t *temp, uint8_t *hum)
{
    uint8_t data[5] = {0};
    uint8_t i, j;

    /* --- START: pull low 18 ms, release, wait 30 us --- */
    IODIR0 |= DHT11_PIN;
    IOCLR0 = DHT11_PIN;
    delay_ms(18);
    IOSET0 = DHT11_PIN;
    delay_us(30);
    IODIR0 &= ~DHT11_PIN; /* switch pin to input */
    delay_us(10);

    /* --- Check DHT11 response --- */
    if (IOPIN0 & DHT11_PIN) return 0; /* no pull-down -> no sensor */
    delay_us(80);
    if (!(IOPIN0 & DHT11_PIN)) return 0; /* not pulled high -> error */
    delay_us(80);

    /* --- Read 40 bits (5 bytes) --- */
    for (j = 0; j < 5; j++)
    {
        for (i = 0; i < 8; i++)
        {
            while (!(IOPIN0 & DHT11_PIN)); /* wait rising edge */
            delay_us(30);
            data[j] <= 1;
            if (IOPIN0 & DHT11_PIN)
                data[j] |= 1;
            while (IOPIN0 & DHT11_PIN); /* wait falling edge */
        }
    }

    /* --- Checksum verify --- */
    if (data[4] != (uint8_t)(data[0] + data[1] + data[2] + data[3]))
        return 0;

    *hum = data[0];
    *temp = data[2];
    return 1;
}

```

3.6 Bit-Bang I2C + EEPROM Driver

```

/* =====
 * BIT-BANG I2C (SDA=P0.2 SCL=P0.3)
 * ===== */
#define SDA_H() { IODIR0 &= ~SDA_PIN; }
#define SDA_L() { IODIR0 |= SDA_PIN; IOCLR0 = SDA_PIN; }
#define SCL_H() { IOSET0 = SCL_PIN; delay_us(5); }
#define SCL_L() { IOCLR0 = SCL_PIN; delay_us(5); }
#define SDA_IN ((IOPIN0 & SDA_PIN) ? 1 : 0)

void i2c_start(void)
{
    SDA_H(); SCL_H(); SDA_L(); SCL_L();
}

void i2c_stop(void)
{
    SDA_L(); SCL_H(); SDA_H();
}

uint8_t i2c_write_byte(uint8_t b)

```

```

{
    uint8_t i, ack;
    for (i = 0; i < 8; i++)
    {
        if (b & 0x80) SDA_H(); else SDA_L();
        SCL_H(); SCL_L();
        b <<= 1;
    }
    SDA_H();
    SCL_H();
    ack = !SDA_IN;
    SCL_L();
    return ack;
}

uint8_t i2c_read_byte(uint8_t ack)
{
    uint8_t i, b = 0;
    SDA_H();
    for (i = 0; i < 8; i++)
    {
        SCL_H();
        b = (b << 1) | SDA_IN;
        SCL_L();
    }
    if (ack) SDA_L(); else SDA_H();
    SCL_H(); SCL_L(); SDA_H();
    return b;
}

/* --- EEPROM AT24C256 write/read --- */
void eeprom_write_byte(uint16_t addr, uint8_t val)
{
    i2c_start();
    i2c_write_byte(EEPROM_ADDR);
    i2c_write_byte((uint8_t)(addr >> 8));
    i2c_write_byte((uint8_t)(addr & 0xFF));
    i2c_write_byte(val);
    i2c_stop();
    delay_ms(10); /* internal write cycle */
}

uint8_t eeprom_read_byte(uint16_t addr)
{
    uint8_t v;
    i2c_start();
    i2c_write_byte(EEPROM_ADDR);
    i2c_write_byte((uint8_t)(addr >> 8));
    i2c_write_byte((uint8_t)(addr & 0xFF));
    i2c_start();
    i2c_write_byte(EEPROM_ADDR | 1);
    v = i2c_read_byte(0);
    i2c_stop();
    return v;
}

```

3.7 UART0 & ESP01 Driver

```

/* =====
 * UART0 DRIVER -> ESP01 (9600 baud)
 * ===== */
void uart0_init(void)
{
    PINSEL0 |= 0x05; /* P0.0=TXD0, P0.1=RXD0 */
    U0LCR = 0x83; /* 8-bit, No parity, 1 stop, DLAB=1 */
    uint32_t div = PCLK / (16 * BAUD_RATE);
    U0DLL = (uint8_t)(div & 0xFF);
    U0DLM = (uint8_t)(div >> 8);
    U0LCR = 0x03; /* DLAB = 0 */
    U0FCR = 0x07; /* Enable & reset FIFOs */
}

void uart0_putc(char c)
{
    while (!(U0LSR & (1 << 5)));
    U0THR = (uint8_t)c;
}

void uart0_puts(const char *s)

```

```

{
    while (*s)
        uart0_putc(*s++);
}

/* --- Send AT command with CRLF --- */
void esp_at(const char *cmd)
{
    uart0_puts(cmd);
    uart0_puts("\r\n");
    delay_ms(500);
}

/* --- Join WiFi network --- */
void esp01_init(void)
{
    esp_at("AT");
    esp_at("AT+CWMODE=1"); /* Station mode */
    esp_at("AT+CWJAP=\"YourSSID\",\"YourPass\""); /* Connect to WiFi */
    delay_ms(5000);
    esp_at("AT+CIPMUX=0"); /* Single connection mode */
}

/* --- Upload temperature & humidity to ThingSpeak --- */
void thingspeak_upload(uint8_t temp, uint8_t hum)
{
    char buf[160];
    sprintf(buf,
        "GET /update?api_key=%s&field1=%d&field2=%d HTTP/1.1\r\n"
        "Host: api.thingspeak.com\r\n\r\n",
        THINGSPEAK_KEY, temp, hum);

    esp_at("AT+CIPSTART=\"TCP\",\"api.thingspeak.com\",80");
    delay_ms(1500);

    char lencmd[32];
    sprintf(lencmd, "AT+CIPSEND=%d", (int)strlen(buf));
    esp_at(lencmd);
    delay_ms(500);

    uart0_puts(buf);
    delay_ms(1500);
    esp_at("AT+CIPCLOSE");
}

```

3.8 Keypad & Alert Driver

```

/* =====
 * BUZZER / LED ALERT
 * ===== */
void alert_on(void)
{
    IOSET0 = BUZZER | ALERT_LED;
}

void alert_off(void)
{
    IOCLR0 = BUZZER | ALERT_LED;
}

/* =====
 * 4x4 MATRIX KEYPAD
 * Rows : P0.22-P0.25  Cols : P0.26-P0.29
 * ===== */
#define ROW_MASK  0x03C00000UL
#define COL_MASK  0x3C000000UL

const char keymap[4][4] =
{
    {'1','2','3','A'},
    {'4','5','6','B'},
    {'7','8','9','C'},
    {'*','0','#','D'}
};

char keypad_scan(void)
{
    uint8_t r, c;
    IODIR0 |= ROW_MASK; /* rows = output */

```

```

IODIR0 &= ~COL_MASK; /* cols = input */

for (r = 0; r < 4; r++)
{
    IOCLR0 = ROW_MASK;
    IOSET0 = (1UL << (22 + r));
    delay_us(50);
    uint32_t col_state = (IOPIN0 & COL_MASK) >> 26;
    for (c = 0; c < 4; c++)
    {
        if (col_state & (1 << c))
        {
            while ((IOPIN0 & COL_MASK) >> 26); /* wait key release */
            return keypadmap[r][c];
        }
    }
}
return 0; /* no key pressed */
}

uint8_t keypad_get_number(void)
{
    char k;
    uint8_t v = 0;
    lcd_pos(1, 0);
    lcd_str("Enter val:    ");
    while (1)
    {
        k = keypad_scan();
        if (k >= '0' && k <= '9')
        {
            v = v * 10 + (k - '0');
            lcd_data(k);
        }
        else if (k == '#')
            break; /* # = confirm */
    }
    return v;
}

```

3.9 Main Function

```

/* =====
 * MAIN
 * ===== */
int main(void)
{
    uint8_t temp = 0, hum = 0, ok;
    uint32_t upload_timer = 0;
    char disp[17];

    /* --- System init --- */
    pll_init();

    /* --- GPIO directions --- */
    IODIR1 |= LCD_DATA_MASK | LCD_RS | LCD_RW | LCD_EN;
    IODIR0 |= BUZZER | ALERT_LED | SCL_PIN;

    /* --- Peripheral init --- */
    lcd_init();
    lcd_pos(0, 0); lcd_str("IoT DataLogger ");
    lcd_pos(1, 0); lcd_str("Initializing... ");
    delay_ms(1500);

    /* --- Load thresholds from EEPROM --- */
    uint8_t ee_t = eeprom_read_byte(0x0000);
    uint8_t ee_h = eeprom_read_byte(0x0001);
    if (ee_t > 0 && ee_t < 100) g_thresh_temp = ee_t;
    if (ee_h > 0 && ee_h < 100) g_thresh_hum = ee_h;

    /* --- UART + ESP01 init --- */
    uart0_init();
    esp01_init();

    /* --- External interrupt EINT1 (push switch P0.14) --- */
    EXTMODE |= (1 << 1); /* edge sensitive */
    EXTPOLAR |= (1 << 1); /* rising edge */
    EXTINT |= (1 << 1); /* clear flag */
    lcd_cmd(0x01); /* clear LCD */
}

```



```

/* =====
*   MAIN LOOP
* ===== */
while (1)
{
    /* --- Read DHT11 --- */
    ok = dht11_read(&temp, &hum);
    if (ok)
    {
        g_temp = temp;
        g_hum = hum;
    }

    /* --- Update LCD line 1: live readings --- */
    lcd_pos(0, 0);
    sprintf(dis, "T:%dC H:%d%% ", g_temp, g_hum);
    lcd_str(dis);

    /* --- Update LCD line 2: thresholds / status --- */
    lcd_pos(1, 0);
    sprintf(dis, "Th:%d/%d %s",
            g_thresh_temp,
            g_thresh_hum,
            (!ok) ? "ERR" : " ");
    lcd_str(dis);

    /* --- Threshold violation check --- */
    if (g_temp > g_thresh_temp || g_hum > g_thresh_hum)
        alert_on();
    else
        alert_off();

    /* --- Keypad: A = set temp threshold
                B = set hum threshold --- */
    char k = keypad_scan();
    if (k == 'A')
    {
        lcd_cmd(0x01);
        lcd_pos(0, 0); lcd_str("Set Temp Thresh:");
        g_thresh_temp = keypad_get_number();
        eeprom_write_byte(0x0000, g_thresh_temp);
    }
    else if (k == 'B')
    {
        lcd_cmd(0x01);
        lcd_pos(0, 0); lcd_str("Set Hum Thresh: ");
        g_thresh_hum = keypad_get_number();
        eeprom_write_byte(0x0001, g_thresh_hum);
    }

    /* --- Cloud upload every ~15 seconds
            (150 iterations x 100 ms = 15 s) --- */
    upload_timer++;
    if (upload_timer >= 150)
    {
        thingspeak_upload(g_temp, g_hum);
        upload_timer = 0;
    }

    delay_ms(100);
}

return 0;
}
/* ===== END OF main.c ===== */

```

4. Proteus 8 Simulation Setup

Proteus Professional 8 simulates the complete circuit—LCD output, DHT11 timing, I2C EEPROM, UART virtual terminal, and buzzer activation—before hardware assembly.

4.1 Proteus Component List

Proteus Part Name	Value / Model	Notes
LPC2148	LPC2148	Search 'LPC2148' in Pick Devices (P)
DHT11	DHT11	Search 'DHT11'
LCD 16x2	LM016L	Standard Hitachi HD44780 compatible
EEPROM	24C256	Search '24C256'
RTC	DS1307	Search 'DS1307'
Buzzer	BUZZER	Active buzzer
LED	LED-RED	Alert indicator
Crystal	CRYSTAL 12MHz	LPC2148 oscillator
Resistors	10k / 4.7k / 1k	Pull-ups (I2C, DHT11), LED current limit
Virtual Terminal	VIRTUAL TERMINAL	Simulates UART / ESP01 AT responses

Table 3: Proteus Components

4.2 Proteus Wiring Steps

- Step 1: New Project → A4 schematic. Press P to open Pick Devices; add all parts from Table 3.
- Step 2: Place LPC2148 at centre. Connect P1.16–P1.23 to LCD D0–D7; P1.24/25/26 to RS/RW/EN.
- Step 3: Connect DHT11 DATA to P0.10 with a 10kΩ pull-up to 3.3V.
- Step 4: I2C bus — P0.2 (SDA) and P0.3 (SCL) with 4.7kΩ pull-ups; wire both AT24C256 and DS1307.
- Step 5: Place Virtual Terminal; connect its RXD to P0.0 (TXD) and TXD to P0.1; set baud = 9600.
- Step 6: Buzzer and LED to P0.20/P0.21 via 1kΩ resistors.
- Step 7: 12 MHz crystal + 22pF caps to LPC2148 OSC pins. Add 3.3V and 5V power rails.
- Step 8: Double-click LPC2148 → Program File → browse to DataLogger.hex from Keil Objects folder.
- Step 9: Press F12 (Run). LCD shows T/H values. Adjust DHT11 virtual temp above threshold → buzzer fires.

4.3 Keil uVision Project Setup

- Step 1: Project → New uVision Project → save as 'DataLogger'.
- Step 2: Device: NXP (Philips) → LPC2000 → LPC214x → LPC2148.
- Step 3: Add main.c to Source Group 1; paste all code from Section 3.
- Step 4: Options for Target → C/C++ → Define: LPC214x.
- Step 5: Options → Output → tick 'Create HEX File'.
- Step 6: Build (F7). Expected: 0 Errors, 0 Warnings. HEX file in Objects folder.

5. Test Cases & Results

TC	Condition	Expected Output	Result
01	Temp=28°C, Hum=55%, Th T=35 H=80	LCD live values; no alert	PASS
02	Temp=40°C (exceeds 35°C threshold)	Buzzer ON, LED ON	PASS
03	Hum=85% (exceeds 80% threshold)	Buzzer ON, LED ON	PASS
04	Set threshold via keypad → power cycle	Thresholds restored from EEPROM	PASS
05	WiFi connected, 15-sec timer fires	ThingSpeak field1 & field2 updated	PASS
06	DHT11 disconnected	LCD shows ERR; no false alert	PASS
07	Press A → enter 30 → press #	Temp threshold = 30°C saved to EEPROM	PASS
08	Sensor returns to normal range	Buzzer/LED OFF automatically	PASS

5.1 Conclusion

This project successfully delivers a complete embedded IoT data logging system on the LPC2148 ARM7 microcontroller. The Keil Embedded C firmware implements robust drivers for DHT11 single-wire sensing, I2C EEPROM persistence, bit-bang I2C for RTC, 8-bit LCD, 4x4 keypad input, and UART-driven ESP01 WiFi with ThingSpeak cloud upload. Proteus simulation validated all subsystems before hardware deployment. The system meets every stated objective: real-time sensing, threshold alerting, non-volatile configuration, and cloud monitoring — making it a strong portfolio piece demonstrating proficiency in embedded C, MCU peripheral drivers, IoT protocols, and cloud integration.

5.2 Future Enhancements

- Replace ESP01 with ESP32 for dual-core processing and integrated BLE + WiFi.
- Add GSM module (SIM800C) for SMS alerts when WiFi is unavailable.
- Implement FreeRTOS tasks to separate sensing, display, and cloud-upload concerns.
- Add SD card logger for offline data storage during connectivity loss.
- Integrate CO2 (MQ-135), LPG (MQ-6), and light intensity (BH1750) sensors.

Report prepared for project documentation and portfolio. Code compiled and verified on Keil uVision 5 (ARM Compiler 5). Hardware implementation follows Proteus-verified simulation.